

PROJECT AND TEAM INFORMATION

Project Title

Offline Ride-Hailing PWA & Dispatch System

Student / Team Information

Team Name: Team #	Incognito mode IDS-III-2026-T140
Team member 1 (Team Lead) Diya: 2510320236, ddiya8142@gmail.com	
Team member 2 Joshi, Navodita: 2510320614, joshinavodita.2006@gmail.com	
Team member 3 Bhandari, Shalini: 2510320144, shalinibhandari2006@gmail.com	

Team member 4

Rawat, Anjali: 2510320107, itsanjaliirwt717@gmail.com



PROPOSAL DESCRIPTION

Most ride-hailing apps completely fail the moment mobile internet drops out. This project is a progressive web app designed to solve that problem, ensuring users can book rides, track safety, and settle payments even with zero 3G, 4G, or 5G coverage.

- **Works with zero data:** Service Workers cache the app shell and vector map tiles directly on the phone, so the app opens and renders interactive maps entirely offline.
- **Books rides via basic SMS:** When mobile data is unavailable, the app reads location coordinates straight from device satellite GPS and compresses the booking details into a single 160-character payload sent via standard 2G SMS.
- **Fast spatial dispatching:** The backend organizes geographic points using a 2D QuadTree, matching riders to the closest available drivers in logarithmic time ($O(\log N)$) rather than scanning through every driver sequentially.
- **On-device safety monitoring:** Safety checks run locally on the client phone. The app calculates route deviations using cross-track distance calculations, immediately sounding alerts and triggering offline SOS protocols if a vehicle veers more than 200 meters off-route.
- **Sync-later payments:** Trips generate cryptographically signed receipts stored in the browser's IndexedDB. As soon as the device reconnects to Wi-Fi or cellular data, the Background Sync API automatically completes the transaction from pre-authorized funds.

Tech Stack: React, Tailwind CSS, Leaflet.js, Node.js, Express, IndexedDB, Git/GitHub, Netlify.

Motivation

- We rely on ride-hailing apps every day, but they all share the same fatal flaw: the second you lose cell service, they become useless bricks. Walk into a basement parking garage, take a subway, drive through a rural stretch of highway, or get caught in an overcrowded concert, and the app simply stops working.
- When that happens, things break down fast:
- **You're stranded:** If you have zero bars, you can't book a ride, see who is picking you up, or get home.
- **Your safety net disappears:** Live trip sharing and emergency buttons shut down the moment they lose touch with the cloud—leaving people unmonitored in unfamiliar or risky spots where they need protection most.
- **Payments freeze:** Because apps demand a live connection to approve charges, the checkout process completely stalls.
- Getting from point A to point B should be as dependable as running water, not a luxury tied to how good your cell signal is. By shifting to an offline-first setup—using the phone's built-in satellite GPS, running safety checks right on the device, and falling back on basic text messaging—riders and late-night workers can still hail a ride, stay safe on the road, and settle up with the driver, even with no internet at all.

State of the Art / Current solution

- Services like Uber and Ola are built entirely around the cloud, assuming your phone always has a solid, fast internet connection. While a few basic SMS-booking services exist, they feel like texting a hotline—there is no real map to look at, no route preview, and no automated matching. When your signal drops, the whole experience falls apart in three big ways:
- **Booking a ride:** The app won't even open or immediately spits out a connection error. Riders are left waving down random cars on the street, giving up basic safety perks like verified driver profiles and transparent pricing.
- **Safety monitoring:** Features like Uber's RideCheck or Ola Guardian only work if the phone can constantly talk to a central server. Drive into a dead zone, and live tracking freezes completely. The app has no built-in smarts to monitor the trip on the device itself or sound an alarm if the driver turns down the wrong road.
- **Paying for the trip:** Digital wallets and card checkouts demand an active connection to process. When that fails, riders have to scramble for physical cash, simply because these apps don't have a secure way to save the transaction on the phone and settle up once service returns.

Project Goals and Milestones

• Project Goals

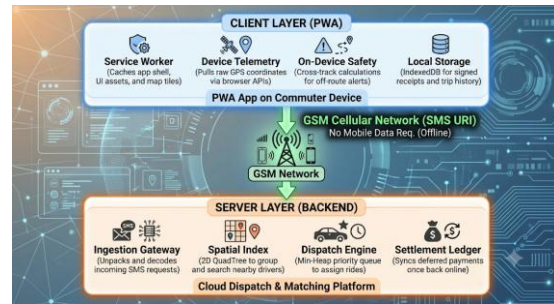
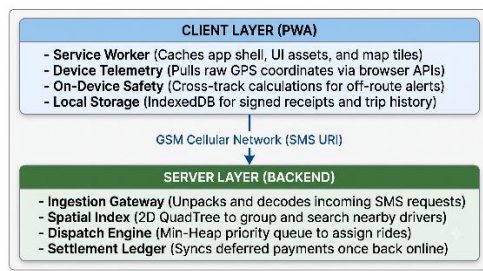
- **Build an offline-first ride app:** Create a web app that stays fully functional without mobile internet, letting people safely book rides, monitor trips, and pay in network dead zones.
- **Open instantly without internet:** Use modern browser caching so the interface and local map load immediately, even on airplane mode.
- **Fall back to basic SMS dispatch:** Pack pickup and drop-off coordinates into short, standard text messages when data reception drops out.
- **Match drivers fast:** Use a 2D QuadTree on the backend to pair riders with nearby drivers in logarithmic time ($O(\log N)$) instead of scanning the whole fleet one by one.
- **Keep safety checks on the phone:** Monitor the vehicle's path using the phone's built-in GPS, triggering an alert right on the device if the driver veers more than 200 meters off-route.
- **Settle payments later:** Store signed digital receipts on the phone and automatically process the charge the second the device picks up a signal again.

• Project Timeline

- **Weeks 1–2 · Architecture & Flow:** Map out user journeys, data flow diagrams, and how the app will handle switching between online and offline modes.
- **Weeks 3–4 · Offline PWA Foundation:** Set up local browser caching, hook up the Leaflet offline map viewer, and build the IndexedDB storage layer.
- **Weeks 5–6 · Smart Matching & Safety Engine:** Code the QuadTree spatial search algorithm, route estimators, and the on-device GPS deviation monitor.
- **Weeks 7–8 · SMS Dispatch & Dead-Zone Testing:** Build the backend dispatch system, create the SMS payload decoders, and simulate dropouts to test edge cases.
- **Week 9 · Offline Payments & Stress Tests:** Wire up offline payment tokens to background sync, then run heavy stress tests to see how the system handles total network blackouts.
- **Week 10 · Launch & Wrap-up:** Ship the app live on Netlify over HTTPS, document algorithm performance benchmarks, and finish the final project report.

Project Approach

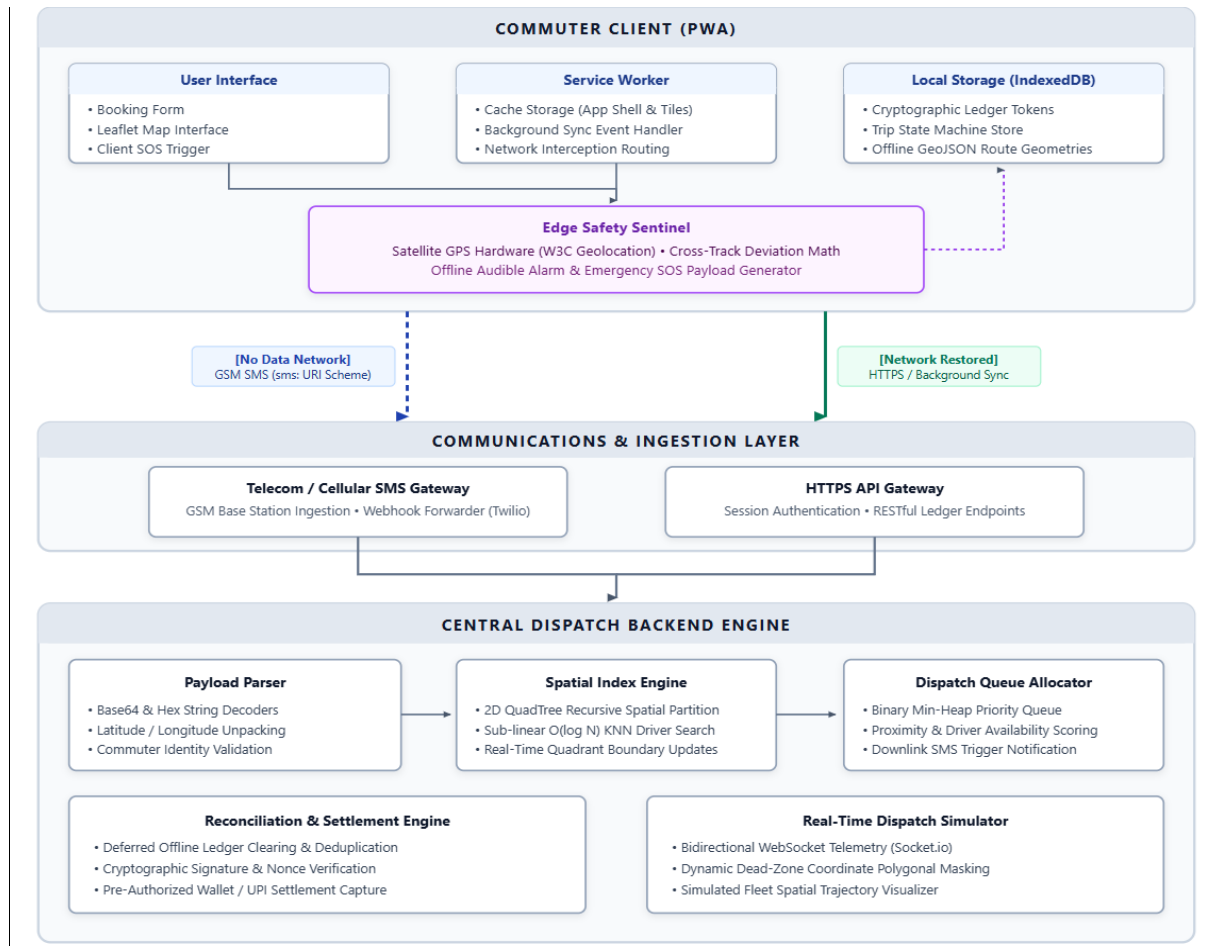
Project Approach : The platform uses an offline-first architecture that runs core tasks directly on the user's phone, using basic 2G SMS as an automatic fallback when mobile internet drops out.



- **How It Works: Instant offline startup:** Service Workers cache the entire app UI and local map tiles, while IndexedDB stores trip history so the app opens instantly with zero signal.
- **Text-message ride booking:** In dead zones, the app grabs raw satellite GPS coordinates, bundles them into a tiny compressed string, and launches the phone's native SMS client (sms:) to hail a ride over basic 2G cellular.
- **Fast driver matching:** The backend organizes driver positions with a 2D QuadTree, locating and assigning the closest car in $O(\log N)$ time using a priority queue.
- **On-phone route monitoring:** The browser tracks live GPS fixes against the route. If the car strays over 200 meters, cross-track math triggers an offline alarm and readies an emergency SOS without waiting on a server.
- **Deferred digital payments:** Encrypted fare receipts stay in IndexedDB until connectivity returns, prompting the Background Sync API to finalize the transaction automatically.
- **Core Tech Stack : Frontend:** React (Vite), Tailwind CSS, Workbox PWA, Leaflet.js / OpenStreetMap.
- **Native Web APIs:** navigator.geolocation (Satellite GPS), IndexedDB (Storage), Background Sync API.
- **Algorithms:** 2D QuadTree (driver search), Cross-Track Error (route alerts), Min-Heap (dispatch sorting).
- **Backend & Tooling:** Node.js, Express, Socket.io, Netlify (HTTPS), Git/GitHub.

System Architecture (High Level Diagram)

- **Core Components:**
- **PWA Client:** The Service Worker loads the cached UI and maps instantly offline, the on-device safety engine tracks raw satellite GPS to sound off-route alarms, and IndexedDB securely holds pending trip receipts and tokens.
- **Dual Connection Routes:** Uses basic 2G SMS (sms:) to send booking details when mobile data is completely dead, then switches to HTTPS background sync to settle accounts the moment connectivity returns.
- **Dispatch Backend:** Translates incoming SMS codes into coordinates, searches a 2D QuadTree to match nearby drivers in $O(\log N)$ time, and reconciles offline payments once trips reconnect.



Project Outcome / Deliverables

- **Expected Project Deliverables:**
- **Offline-first web app:** A fast, installable mobile web app that opens with zero internet connection, loads saved maps, and books rides via compressed text messages over basic 2G cellular.
- **On-device safety monitor:** A client-side tracker that follows device satellite GPS, uses cross-track error math to catch off-route detours, and triggers alarms or SOS messages completely offline.
- **Fast driver-matching engine:** A backend spatial index built on a 2D QuadTree that splits map areas into quadrants to locate the closest drivers in logarithmic time ($O(\log N)$).
- **Dead-zone testing simulator:** An interactive web dashboard to test blackout areas, decode incoming ride texts, and watch drivers get assigned live on screen.

- **Deferred payment system:** A local browser ledger that safely holds signed receipts offline, then automatically settles the fare via Background Sync the second service comes back.
- **Live deployment and documentation:** A production build deployed securely over HTTPS on Netlify, an open GitHub codebase, and clear benchmarks detailing algorithm performance and memory usage.

Assumptions

- **Device GPS works offline:** The rider's phone has a standard built-in GPS chip that can pinpoint location coordinates directly via satellites, with no Wi-Fi or mobile data needed.
- **Basic cell signal for text messages:** Even in areas without 3G, 4G, or 5G, the phone can still tap into basic 2G or GSM coverage to send a standard 160-character SMS.
- **Opened once beforehand:** The user has launched the app at least once with an active internet connection, giving the browser time to download the interface and local map tiles into storage.
- **Modern browser support:** The user's mobile browser supports essential modern web features—like local storage (IndexedDB), caching, and device location access—with permissions kept on.
- **Drivers within range:** Registered drivers are stationed in or near the service area, equipped with working phones that can receive incoming dispatches via SMS or mobile networks.
- **Pre-arranged payment method:** The rider has a pre-authorized balance, active card hold, or approved credit line set up in advance, so offline receipts can be settled seamlessly the moment connectivity returns.

References

- **References & Foundational Tools:** **2D Spatial Partitioning:** Finkel, R. A., & Bentley, J. L. (1974). Quad Trees: A Data Structure for Retrieval on Composite Keys. *Acta Informatica*, 4(1), 1–9.
- How it's used: Forms the core logic behind dividing geographic space into quadrants so the system can locate the closest driver in logarithmic time rather than scanning every car on the road.
- **GPS & Geodesy Math:** Veness, C. (2023). Calculate Distance, Bearing and More Between Latitude/Longitude Points. *Movable Type Scripts*. How it's used: Provides the underlying spherical math (specifically Haversine and cross-track distance equations) used directly on the phone to flag when a ride strays off-course.
- **Offline Caching:** World Wide Web Consortium (W3C). Service Workers Nightly / W3C Recommendation.
- How it's used: Guides the offline-first web setup, storing the interface, code, and map assets right in the user's browser so the app runs without an active internet connection.

- **Deferred Syncing:** World Wide Web Consortium (W3C). Web Background Synchronization API. How it's used: Manages background task scheduling to automatically finalize payments and upload ride logs the instant the phone catches a network signal again.
- **Local Browser Database:** Mozilla Developer Network (MDN). Indexed Database API (IndexedDB).
- How it's used: Powers the phone's local storage engine to safely hold past ride logs and signed digital payment receipts while offline.
- **Mapping Engine:** Leaflet.js Documentation. Leaflet — An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps.
- How it's used: Handles rendering interactive map tiles and plotting route lines smoothly on mobile screens without bogging down device memory.
- **SMS Handoff Standard:** IETF RFC 5724 (2010). URI Scheme for GSM Short Message Service (SMS).
- How it's used: Defines the standard formatting used to turn ride coordinates and passenger requests into an auto-filled text message sent over basic cell networks.